

GameFlowManager: GameState

i Для управления циклом всей игры необходим некий глобальный менеджер, который будет содержать “кнопки” для управления большинством других менеджеров. Тот, кто будет знать текущее состояние игры и менять их, сохраняя логику в норме.

GameFlowManager знает о текущем состоянии игры и сам же контролирует эти состояния. Разберём цикл всех состояний:

1. LoadStage() - именно этот метод срабатывает при нажатии старт в главном меню. Он подгружает случайный уровень, сбрасывает кошелек, устанавливает текущий этаж в менеджере сложности, показывает HUD, сообщает спавнеру, что нужно инициализироваться. Появляется кнопка “Нажмите чтобы начать”.
2. StartPlaying() - запускается при нажатии на эту кнопку. Начинается сама игра.
3. EscMenu() - запускается при нажатии кнопки на HUD, либо по нажатию на ESC.
4. LeaveGame() - срабатывает по нажатию кнопки выхода и следом вызывает MenuGame()
5. MenuGame() - выкидывает нас на главное меню.
6. ReturnPlaying() - ещё одна нажатие ESC или нажатие на кнопку return - продолжает игру.
7. DefeatGame() - в случае если здоровье базы опустилось до 0, то мы проиграли. У нас открывает DeathScreen, где мы можем - попробовать снова или покинуть игру.
8. RestartGame() - кнопка на экране проигрыша - перезапускает игру заного.
9. OnStageCompleted() - победа на этаже - запускает окно выбора артефакта.
10. ContinueAfterArtifact() - перейти в следующий этаж.

```
1 using System;
2 using UnityEngine;
3 using UnityEngine.InputSystem;
4 using UnityEngine.SceneManagement;
5
6 public enum GameState
7 {
8     Menu,
9     Loading,
10    Playing,
11    ArtifactChoice,
12    Defeat,
13    Prestart,
14    EscMenu
15 }
16
17
18 public class GameFlowManager : Manager<GameFlowManager>
```

```

19 {
20     public GameState State { get; private set; } = GameState.Menu;
21
22     private bool restartLocked = false;
23
24     private void Start()
25     {
26         BaseManager.Instance.OnBaseDestroyed += Defeat;
27     }
28
29     private void OnDisable()
30     {
31         BaseManager.Instance.OnBaseDestroyed -= Defeat;
32     }
33
34     public void StartRun()
35     {
36         GameManager.Instance.AddFloor();
37         LoadStage();
38     }
39
40     public void OnStageCompleted()
41     {
42         State = GameState.ArtifactChoice;
43         SpeedGameManager.Instance.Pause();
44         HudManager.Instance.ShowStageCompletedCanvas();
45     }
46
47     public void ContinueAfterArtifact()
48     {
49         GameManager.Instance.AddFloor();
50         LoadStage();
51     }
52
53     private void LoadStage()
54     {
55         if (State == GameState.Loading) return;
56         State = GameState.Loading;
57
58         AudioManager.Instance.StopMusic();
59
60         SceneLoader.Instance.LoadScene(
61             LevelManager.Instance.GetRandomLevel(),
62             () =>
63             {
64                 restartLocked = false;
65
66                 DifficultyManager.Instance.SetFloor(GameManager.Instance.Floor);
67                 WalletManager.Instance.ResetWallet();
68                 BaseManager.Instance.ResetBase();
69                 HudManager.Instance.Reset();
70                 HudManager.Instance.Show();
71                 State = GameState.Prestart;
72                 EnemySpawnerManager.Instance.StopSpawning();
73                 EnemySpawnerManager.Instance.SceneEnabled(5); // Если
74             },
75             () => { HudManager.Instance.ShowPrestartButton(); }
76         );
77     }
78
79     public void StartPlaying()
80     {
81         if (State != GameState.Prestart) return;
82
83         State = GameState.Playing;
84         HudManager.Instance.HidePrestart();
85         GameManager.Instance.StartGame();

```

```

86         SpeedGameManager.Instance.Resume();
87         AudioManager.Instance.PlayMusicType(MusicType.FightRandom);
88
89         EnemySpawnerManager.Instance.StartSpawning();
90     }
91
92     public void Defeat()
93     {
94         State = GameState.Defeat;
95
96         SpeedGameManager.Instance.Pause();
97         HudManager.Instance.ShowDefeatScreen();
98         GameManager.Instance.EndTimer();
99         EnemySpawnerManager.Instance.StopSpawning();
100    }
101
102    public void RestartGame()
103    {
104        if (restartLocked) return;
105        if (State != GameState.Defeat) return;
106
107        restartLocked = true;
108        GameManager.Instance.EndRun();
109        ArtefactManager.Instance.ResetInventory();
110        EnemySpawnerManager.Instance.StopSpawning();
111        LoadStage();
112    }
113
114    public void LeaveGame()
115    {
116        if (State == GameState.Loading) return;
117        //HudManager.Instance.HideDefeatScreen();
118        GameManager.Instance.EndRun();
119        ArtefactManager.Instance.ResetInventory();
120        EnemySpawnerManager.Instance.StopSpawning();
121        MenuGame();
122    }
123
124    public void MenuGame()
125    {
126        State = GameState.Loading;
127
128        SpeedGameManager.Instance.Pause();
129        EnemySpawnerManager.Instance.StopSpawning();
130
131        SceneLoader.Instance.LoadScene("MainMenu",
132            () =>
133            {
134                HudManager.Instance.Hide();
135                State = GameState.Menu;
136                AudioManager.Instance.PlayMusicType(MusicType.Menu);
137            }
138        );
139    }
140
141    public void EscMenu()
142    {
143        State = GameState.EscMenu;
144        SpeedGameManager.Instance.Pause();
145    }
146
147    public void ReturnPlaying()
148    {
149        State = GameState.Playing;
150        SpeedGameManager.Instance.SetPastSpeed();
151    }

```

☐ Многие менеджеры получают доступ о текущем состоянии. Например монетки не будут падать, если текущее состояние не Playing. Соответственно, при выборе артефакта, на экране смерти, в меню - монетки не будут расти. Также дело обстоит с управлением временем через клавиатуру.

⚠ В скрипте на этапе написания документации найдены некоторые оплошности и ожидаются баги. Вскоре они будут исправлены!